

Metavault

A self-hostable universal media library

Nikoloz Kvrivishvili, Guga Akopashvili, Giorgi Ratiani



ILIA STATE UNIVERSITY
Faculty of Business, Technology and Education
School of Technology
Computer Science Undergraduate Program

SUPERVISOR
Prof. Dr. Erekle Magradze



Metavault is a self-hostable web application for tracking movies, TV shows, anime, games, books, manga, and other media in one place. The core library works without external services: users can create records, search them, organize them, and export their data. Source integrations and AI are opt-in. TMDb, AniList, IGDB, or OpenLibrary can fill metadata, and a local or hosted OpenAI-compatible model can power chat and recommendations.



Open Metavault

MOTIVATION AND PRODUCT SCOPE

Purpose of Metavault

People who track several media types usually split their history across separate tools. Ratings, statuses, tags, unfinished items, and backups end up spread across different accounts.

Opt-in integrations

The library, EZQ, import/export, and account flows do not require metadata sources or an AI provider. Enrichment and assistant features are toggled on by users explicitly.

Data ownership

Library data lives in SQLite. It can be easily exported in bundles in a portable format, so it can be backed up, inspected, or moved.

Shared media model

Movies, shows, anime, games, books, and manga share one record shape for status, ratings, tags, and collections.

Easy enrichment

Configured source APIs can fill titles, covers, dates, public ratings, source IDs, and tags when the user asks for enrichment.

Portable setup

Docker and the open repository make the project easier to run, audit, modify, and reuse.

Features of Metavault

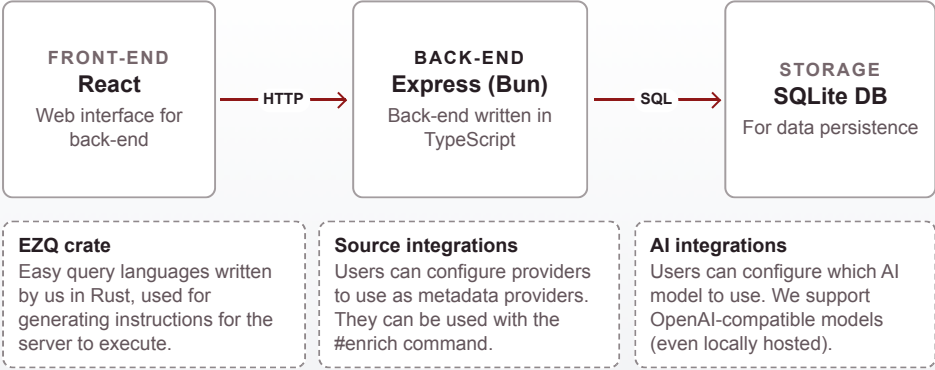
The current app supports the main flows: account creation and management, library management, querying and executing commands with EZQ, entry enrichment, chatting with assistants, getting recommendations, and importing/exporting.

Accounts and access Register, verify email, sign in, update profile, upload avatar, reset password, and delete the account.	Media library Create, view, update, delete, rate, tag, upload covers, and track status across supported media types.
Organization Collections and content nodes for lists, episodes, chapters, volumes, and related links.	Search and EZQ One input for search, create, update, delete, aliases, enrichment, and catalogue pull commands.
Source enrichment Use configured TMDb, AniList, IGDB, or OpenLibrary sources to fill missing metadata.	Recommendations Generate suggestions from catalogue entries, ranked with embeddings, filters, and scoring rules.
Portability Self-host with Docker, persist data in a SQLite database, and move data through import/export bundles.	AI assistant Use an OpenAI-compatible local or hosted model for chat, discuss query results, and get recommendations.

SYSTEM DESIGN AND QUERY MODEL

Overview of Architecture

The web client is the primary interface to our server. The server handles all of the functionality and executes instructions generated by EZQ. SQLite is used to persist the data.



EZQ (Easy Query)

EZQ is a Rust crate created by us. It gives us the ability to execute most common library management operations with easy syntax.

<code>/s status:in_progress type:movie</code>	Search in-progress movies
<code>/c Dune type:book status:planning</code>	Create a book entry
<code>/u Joker > status:Finished</code>	Update entry progress
<code>/d public_rating:<7</code>	Delete entries

Why EZQ matters: EZQ lets users manage their whole library through a single input. It also lets them combine common operations in ways conventional interface-driven querying and filtering systems usually cannot.

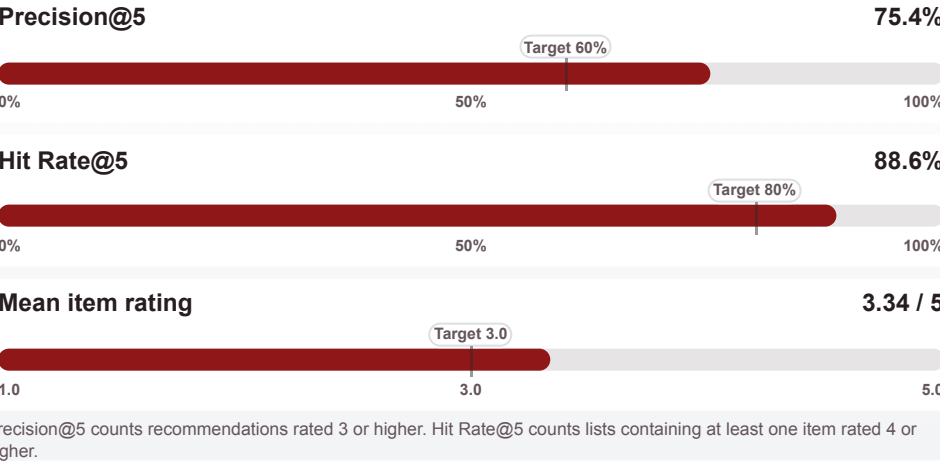
AI AND WORK DISTRIBUTION

Recommendation & AI system

Metavault can connect to user-configured AI models to help with library searches, results, and recommendations. When a user asks for suggestions, the system finds suitable catalogue options, ranks them, and lets the assistant present them.

Study overview

Seven participants evaluated five prompts each, with five recommendations per prompt. These 175 item ratings were used to calculate the evaluation metrics below.



Work distribution

CONDENSED 15-WEEK TIMELINE

We condensed our 15-week timeline into four high-level phases and highlighted each member's corresponding contributions below.

Phase DURATION	Nikoloz Kvrivishvili	Guga Akopashvili	Giorgi Ratiani
Foundation and planning 3 weeks	Repository and CI/CD, EZQ and integration planning	Express server and SQL table schemas	React setup along with core libraries
Central functionality 5 weeks	EZQ crate, build setup for server and client	Authentication and CRUD flows for library, collections, import/export	Web interfaces for library, collections, import/export flows
MVP finalization 5 weeks	Source integrations, command, assistant, recommendation logic	Server integration of the mentioned logic	Client integration for the mentioned logic
Refinement 2 weeks	Fixing EZQ bugs and gaps	Back-end cleanup and test improvement	UI polish and integration improvements